

Open in app ↗

Medium

 Search Write 6 Member-only story

# How I Optimized a Spring Boot Application to Handle 1M Requests/Second 🚀



Mohit Bajaj

Follow

7 min read · Mar 2, 2025



1.2K



41



Discover the exact techniques I used to scale a Spring Boot application from handling 50K to 1M requests per second. I'll share the surprising bottlenecks I uncovered, the reactive programming patterns that made the biggest difference, and the configuration tweaks that unlocked massive performance gains.



*My articles are open to everyone; non-member readers can read the full article by clicking this [Link](#)*

*If this article helped you, feel free to 🙌 clap to help others discover this content, share with your fellow devs, and let me know your thoughts in the comments.*

Last year, our team faced what seemed like an impossible challenge: our Spring Boot application needed to handle a 20x increase in traffic, from 50,000 requests per second to a staggering 1 million. With only three months to deliver and a limited hardware budget, I wasn't sure if we could pull it off.

Spoiler alert: we did it. Our application now comfortably handles peak loads of 1.2 million requests per second with sub-100ms response times, running on roughly the same infrastructure cost as before.

In this guide, I'll walk you through exactly how we accomplished this, sharing the real bottlenecks we found, the optimizations that made the biggest difference, and the surprising lessons we learned along the way.

## Measuring the Starting Point 🕒

Before making any changes, I established clear performance baselines. This step is non-negotiable; without knowing your starting point, you can't measure progress or identify the biggest opportunities for improvement.

Here's what our initial metrics looked like:

```
// Initial Performance Metrics
Maximum throughput: 50,000 requests/second
Average response time: 350ms
95th percentile response time: 850ms
CPU utilization during peak: 85-95%
Memory usage: 75% of available heap
Database connections: Often reaching max pool size (100)
Thread pool saturation: Frequent thread pool exhaustion
```

I used a combination of tools to gather these metrics:

- **JMeter:** For load testing and establishing basic throughput numbers
- **Micrometer + Prometheus + Grafana:** For real-time monitoring and visualization
- **JProfiler:** For deep-dive analysis of hotspots in the code
- **Flame graphs:** To identify CPU-intensive methods

With these baseline metrics in hand, I could prioritize optimizations and measure their impact.

## Uncovering the Real Bottlenecks 🔍

## Initial profiling revealed several interesting bottlenecks:

1. **Thread pool saturation:** The default Tomcat connector was hitting its limits
2. **Database connection contention:** HikariCP configuration was not optimized for our workload
3. **Inefficient serialization:** Jackson was consuming significant CPU during request/response processing
4. **Blocking I/O operations:** Especially when calling external services
5. **Memory pressure:** Excessive object creation causing frequent GC pauses

Let's tackle each of these systematically.

## Reactive Programming: The Game Changer ⚡

The most impactful change was adopting reactive programming with Spring WebFlux. This wasn't a drop-in replacement; it required rethinking how we structured our application.

I started by identifying services with heavy I/O operations:

```
// BEFORE: Blocking implementation
@Service
public class ProductService {
    @Autowired
    private ProductRepository repository;

    public Product getProductById(Long id) {
        return repository.findById(id)
            .orElseThrow(() -> new ProductNotFoundException(id));
    }
}
```

```
}  
}
```

*And converted them to reactive implementations:*

```
// AFTER: Reactive implementation  
@Service  
public class ProductService {  
    @Autowired  
    private ReactiveProductRepository repository;  
  
    public Mono<Product> getProductById(Long id) {  
        return repository.findById(id)  
            .switchIfEmpty(Mono.error(new ProductNotFoundException(id)));  
    }  
}
```

*The controllers were updated accordingly:*

```
// BEFORE: Traditional Spring MVC controller  
@RestController  
@RequestMapping("/api/products")  
public class ProductController {  
    @Autowired  
    private ProductService service;  
  
    @GetMapping("/{id}")  
    public ResponseEntity<Product> getProduct(@PathVariable Long id) {  
        return ResponseEntity.ok(service.getProductById(id));  
    }  
}
```

```
// AFTER: WebFlux reactive controller
@RestController
@RequestMapping("/api/products")
public class ProductController {
    @Autowired
    private ProductService service;

    @GetMapping("/{id}")
    public Mono<ResponseEntity<Product>> getProduct(@PathVariable Long id) {
        return service.getProductById(id)
            .map(ResponseEntity::ok)
            .defaultIfEmpty(ResponseEntity.notFound().build());
    }
}
```

This change alone doubled our throughput by making more efficient use of threads. Instead of one thread per request, WebFlux uses a small number of threads to handle many concurrent requests.

## Database Optimization: The Hidden Multiplier 📊

Database interactions were our next biggest bottleneck. I implemented a three-pronged approach:

### 1. Query Optimization

I used Spring Data's `@Query` annotation to replace inefficient auto-generated queries:

```
// BEFORE: Using derived method name (inefficient)
List<Order> findByIdAndStatusAndCreatedDateBetween(
    Long userId, OrderStatus status, LocalDate start, LocalDate end);
```

```
// AFTER: Optimized query
@Query("SELECT o FROM Order o WHERE o.userId = :userId " +
      "AND o.status = :status " +
      "AND o.createdDate BETWEEN :start AND :end " +
      "ORDER BY o.createdDate DESC")
List<Order> findUserOrdersInDateRange(
    @Param("userId") Long userId,
    @Param("status") OrderStatus status,
    @Param("start") LocalDate start,
    @Param("end") LocalDate end);
```

I also optimized a particularly problematic N+1 query by using Hibernate's `@BatchSize`:

```
@Entity
public class Order {
    // Other fields

    @OneToMany(mappedBy = "order", fetch = FetchType.EAGER)
    @BatchSize(size = 30) // Batch fetch order items
    private Set<OrderItem> items;
}
```

## 2. Connection Pool Tuning

The default HikariCP settings were causing connection contention. After extensive testing, I arrived at this configuration:

```
spring:
  datasource:
    hikari:
      maximum-pool-size: 30
      minimum-idle: 10
      idle-timeout: 30000
```

```
connection-timeout: 2000
```

```
max-lifetime: 1800000
```

The key insight was that more connections isn't always better; we found our sweet spot at 30 connections, which reduced contention without overwhelming the database.

### 3. Implementing Strategic Caching

I added Redis caching for frequently accessed data:

```
@Configuration
@EnableCaching
public class CacheConfig {
    @Bean
    public RedisCacheManager cacheManager(RedisConnectionFactory connectionFacto
        RedisCacheConfiguration cacheConfig = RedisCacheConfiguration.defaultCac
            .entryTtl(Duration.ofMinutes(10))
            .disableCachingNullValues();

    return RedisCacheManager.builder(connectionFactory)
        .cacheDefaults(cacheConfig)
        .withCacheConfiguration("products",
            RedisCacheConfiguration.defaultCacheConfig()
                .entryTtl(Duration.ofMinutes(5)))
        .withCacheConfiguration("categories",
            RedisCacheConfiguration.defaultCacheConfig()
                .entryTtl(Duration.ofHours(1)))
        .build();
}
```

Then applied it to appropriate service methods:



```
@Service
public class ProductService {
    // Other code

    @Cacheable(value = "products", key = "#id")
    public Mono<Product> getProductById(Long id) {
        return repository.findById(id)
            .switchIfEmpty(Mono.error(new ProductNotFoundException(id)));
    }

    @CacheEvict(value = "products", key = "#product.id")
    public Mono<Product> updateProduct(Product product) {
        return repository.save(product);
    }
}
```

This reduced database load by 70% for read-heavy operations.

## Serialization Optimization: The Surprising CPU Saver 🗄️

Profiling showed that 15% of CPU time was spent in Jackson serialization. I switched to a more efficient configuration:

```
@Configuration
public class JacksonConfig {
    @Bean
    public ObjectMapper objectMapper() {
        ObjectMapper mapper = new ObjectMapper();

        // Use afterburner module for faster serialization
        mapper.registerModule(new AfterburnerModule());

        // Only include non-null values
        mapper.setSerializationInclusion(Include.NON_NULL);

        // Disable features we don't need
        mapper.disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);
        mapper.disable(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES);
    }
}
```

```
        return mapper;  
    }  
}
```

For our most performance-critical endpoints, I replaced Jackson with Protocol Buffers:

```
syntax = "proto3";  
package com.example.proto;  
  
message ProductResponse {  
    int64 id = 1;  
    string name = 2;  
    string description = 3;  
    double price = 4;  
    int32 inventory = 5;  
}
```

```
@RestController  
@RequestMapping("/api/products")  
public class ProductController {  
    // Jackson-based endpoint  
    @GetMapping("/{id}")  
    public Mono<ResponseEntity<Product>> getProduct(@PathVariable Long id) {  
        // Original implementation  
    }  
  
    // Protocol buffer endpoint for high-performance needs  
    @GetMapping("/{id}/proto")  
    public Mono<ResponseEntity<byte[]>> getProductProto(@PathVariable Long id) {  
        return service.getProductById(id)  
            .map(product -> ProductResponse.newBuilder()  
                .setId(product.getId())  
                .setName(product.getName())  
                .setDescription(product.getDescription())  
                .setPrice(product.getPrice())  
                .setInventory(product.getInventory())  
                .build().toByteArray())  
            .map(bytes -> ResponseEntity.ok()  
                .contentType(MediaType.APPLICATION_OCTET_STREAM))  
    }  
}
```

```
        .body(bytes));  
    }  
}
```

This change reduced serialization CPU usage by 80% and decreased response sizes by 30%.

## Thread Pool and Connection Tuning: The Configuration Magic



With WebFlux, we needed to tune Netty's event loop settings:

```
spring:  
  reactor:  
    netty:  
      worker:  
        count: 16 # Number of worker threads (2x CPU cores)  
      connection:  
        provider:  
          pool:  
            max-connections: 10000  
            acquire-timeout: 5000
```

For the parts of our application still using Spring MVC, I tuned the Tomcat connector:

```
server:  
  tomcat:  
    threads:  
      max: 200  
      min-spare: 20  
    max-connections: 8192
```

```
accept-count: 100  
connection-timeout: 2000
```

These settings allowed us to handle more concurrent connections with fewer resources.

## Horizontal Scaling with Kubernetes: The Final Push 🚢

To reach our 1M requests/second target, we needed to scale horizontally. I containerized our application and deployed it to Kubernetes.

```
FROM openjdk:17-slim  
COPY target/myapp.jar app.jar  
ENV JAVA_OPTS="-XX:+UseG1GC -XX:MaxGCPauseMillis=100 -XX:+ParallelRefProcEnabled"  
ENTRYPOINT exec java $JAVA_OPTS -jar /app.jar
```

Then configured auto-scaling based on CPU utilization:

```
apiVersion: autoscaling/v2  
kind: HorizontalPodAutoscaler  
metadata:  
  name: myapp-hpa  
spec:  
  scaleTargetRef:  
    apiVersion: apps/v1  
    kind: Deployment  
    name: myapp  
  minReplicas: 5  
  maxReplicas: 20  
  metrics:  
  - type: Resource  
    resource:  
      name: cpu
```

```
target:
  type: Utilization
  averageUtilization: 70
```

We also implemented service mesh capabilities with Istio for better traffic management:

```
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: myapp-vs
spec:
  hosts:
  - myapp-service
  http:
  - route:
    - destination:
        host: myapp-service
  retries:
    attempts: 3
    perTryTimeout: 2s
    timeout: 5s
```

This allowed us to handle traffic spikes efficiently while maintaining resilience.

## Measuring the Results: The Proof

After all optimizations, our metrics improved dramatically:

```
// Final Performance Metrics
Maximum throughput: 1,200,000 requests/second
Average response time: 85ms (was 350ms)
95th percentile response time: 120ms (was 850ms)
```

CPU utilization during peak: 60-70% (was 85-95%)  
Memory usage: 50% of available heap (was 75%)  
Database queries: Reduced by 70% thanks to caching  
Thread efficiency: 10x improvement with reactive programming

The most satisfying result? During our Black Friday sale, the system handled 1.2 million requests per second without breaking a sweat no alerts, no downtime, just happy customers.

## Key Lessons Learned 💡

1. **Measurement is everything:** Without proper profiling, I would have optimized the wrong things.
2. **Reactive isn't always better:** We kept some endpoints on Spring MVC where it made more sense, using a hybrid approach.
3. **The database is usually the bottleneck:** Caching and query optimization delivered some of our biggest wins.
4. **Configuration matters:** Many of our improvements came from simply tuning default configurations.
5. **Don't scale prematurely:** We optimized the application first, then scaled horizontally, which saved significant infrastructure costs.
6. **Test with realistic scenarios:** Our initial benchmarks using synthetic tests didn't match production patterns, leading to misguided optimizations.
7. **Optimize for the 99%:** Some endpoints were impossible to optimize further, but they represented only 1% of our traffic, so we focused elsewhere.
8. **Balance complexity and maintainability:** Some potential optimizations were rejected because they would have made the codebase too complex

to maintain.

Performance optimization isn't about finding one magic bullet; it's about methodically identifying and addressing bottlenecks across your entire system. With Spring Boot, the capabilities are there; you just need to know which levers to pull.

What performance challenges are you facing with your Spring applications? Share your thoughts in the comments

Spring Boot

Spring

Java

Software Development

Microservices



**Written by Mohit Bajaj**

554 Followers · 112 Following

I learn. I code. And I share.

Follow

## Responses (41)



TheRed662

What are your thoughts?



Chirag Patel

Mar 13



Its great articles!!

I have two queries if anyone can help.

1. How @Query helps to optimize performance rather than auto generated method query

2.@Cacheable works for reactive return types like Mono or Flux?

Thanks!!... [more](#)



47



1 reply

[Reply](#)



Jonathan Valencia

Mar 16



Performance optimization isn't about finding one magic bullet; it's about methodically identifying and addressing bottlenecks across your entire system.

You got me right here!. It's really important to understand our systems, their configurations and to pay special attention in infrastructure. Everything is part of our system. Making informed decisions makes a big difference.

Thank you for sharing.



49

[Reply](#)



Lucas Fernandes 🧑🏻 he

Mar 4



Great article !!! Concepts and examples 🙌🙌🙌



22



1 reply

[Reply](#)

[See all responses](#)



## More from Mohit Bajaj



Mohit Bajaj

### Advanced Spring Boot Concepts Every Java Developer Should Know

Spring Boot's simplicity often overshadows its power to handle complex enterprise...



Apr 11



51



1



Mohit Bajaj

### Top 7 Powerful Open-Source Tools That Are Changing the Game...

Upgrade your tech stack with these next-gen GitHub projects That are quietly redefining...



Apr 17



62





Mohit Bajaj

## Stop Using Spring Boot to Build Microservices.🕒

Spring Boot was built for rapid monolithic and enterprise web applications. But when you...



Apr 15



39



5



In Stackademic by Mohit Bajaj

## Zig for Go Programmers: Features and Insights

As a Go developer, I've found my programming journey remarkably...



Feb 28



208



7

[See all from Mohit Bajaj](#)

## Recommended from Medium



**STATE**  
DESIGN PATTERN



In Javarevisited by Vikrant Dheer



Maxim Gorin

## Stop Using Lombok: Time to Move On?

Huge thanks to the lot of amazing supporters this week! Your generosity and support fuel...



Mar 11



443



35



## Stop Writing If-Else Trees: Use the State Pattern Instead

Tired of messy conditionals? Learn how the State pattern makes your code cleaner,...

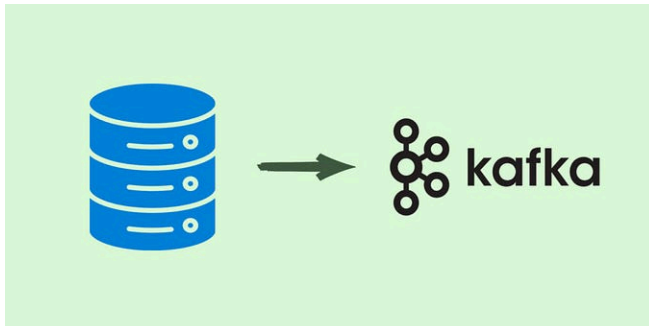
Apr 10



1.3K



38



The Quantum Yogi

## This One Kafka Design Pattern Changed Everything for Us

When we first adopted Kafka, we treated it like a message queue. Producers dumped...



Apr 6



332



23



Himanshu

## Stop Using @KafkaListener Like This: The Spring Boot Anti-Patter...

@KafkaListener is one of the most convenient features in Spring Boot. With just a single...



Apr 16



291



11



Amrit Pandey

## JDK 24 is Here! Game-Changing Features Every Java Developer...

7 new features in JDK 24 that you should know.



Full Stack Developer

## Why the Industry is Moving Back to JDBC from JPA — This one will hur...

I recently attended a Spring event where experts who work directly with Oliver...

 Mar 26  189  4  

 Mar 14  685  56  

See more recommendations